Name: Vansh Jain
Roll No: 124A8036
Class/Div: SE G
Date of Submission:10/02/2026

Experiment No.: 05

Title: Process Management: Synchronization Write a C program to implement solution of Producer consumer problem through Semaphore.

Objective:

- Implement a bounded-buffer Producer–Consumer system using POSIX threads and semaphores. One producer inserts items into a fixed-size circular buffer. One consumer removes items. Use semaphores to handle buffer capacity and a lock for mutual exclusion.
- Worksheet
- POSIX Semaphore Version WITHOUT Mutex (Using Binary Semaphore)

Software: Linux Operating System

Theory: In a multitasking operating system, multiple processes or threads may execute concurrently and share common resources. When such shared resources are accessed without proper control, it may lead to race conditions, causing incorrect or unpredictable results. To avoid this, process synchronization mechanisms are required.

The Producer–Consumer problem is a classical synchronization problem where:

- The producer generates data and places it into a shared buffer.
- The consumer removes data from the buffer and processes it.
- The buffer has a fixed size (bounded buffer).

If the buffer is full, the producer must wait.

If the buffer is empty, the consumer must wait.

In this experiment, POSIX semaphores are used to solve this problem:

- A counting semaphore (empty) tracks the number of empty slots.
- A counting semaphore (full) tracks the number of filled slots.
- A binary semaphore is used to ensure mutual exclusion while accessing the shared buffer, instead of using a mutex.

POSIX threads (pthread) allow concurrent execution of producer and consumer threads, while semaphores (sem_wait and sem_post) coordinate their execution safely. This approach ensures proper synchronization without data inconsistency.

Output/Result:

Section A:

Given / To Use: • Fixed-size buffer (BUFFER_SIZE) • Two indices: in (producer), out (consumer) • Semaphores: empty (empty slots), full (filled slots) • A mutual-exclusion lock for critical section

Algorithm (High-Level):
Initialization:

1. Create buffer[BUFFER_SIZE], set in = 0, out = 0
2. Initialize semaphores:- empty = BUFFER_SIZE- full = 0
3. Initialize mutual exclusion lock
4. Create producer and consumer threads

Producer (repeat forever):

1. Produce an item
2. Wait until an empty slot is available
3. Enter critical section
4. Insert item into buffer[in]
5. Update in = (in + 1) % BUFFER_SIZE
6. Exit critical section
7. Signal that a full slot is available

Consumer (repeat forever):

1. Wait until a full slot is available
2. Enter critical section
3. Remove item from buffer[out]
4. Update out = (out + 1) % BUFFER_SIZE
5. Exit critical section

6. Signal that an empty slot is available
7. Consume the item

Flowchart Hints:

Producer Flow:
START → Produce Item → Is buffer full?
Yes → WAIT
No → Enter CS → Insert at in → Update in → Exit CS → Signal full → REPEAT
Consumer Flow:
START → Is buffer empty?
Yes → WAIT
No → Enter CS → Remove from out → Update out → Exit CS → Signal empty → Consume → REPEAT

Code:

```
GNU nano 7.2                                                    buffer.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 5
#define MAX_ITEMS 4

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item = 1;

    for (int i = 0; i < MAX_ITEMS; i++) {
        printf("Producer produced item: %d\n", item);

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer inserted item %d at index %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        item++;
        sleep(1);
    }

    pthread_exit(NULL);
}

void* consumer(void* arg) {
```

```
GNU nano 7.2                                                    buffer.c *

    for (int i = 0; i < MAX_ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer removed item %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        printf("Consumer consumed item: %d\n", item);
        sleep(1);
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("Program terminated after 4 iterations\n");

    return 0;
}
```

```
siesgst@siesgst-OptiPlex-3000:/home/exp5$ sudo nano buffer.c
siesgst@siesgst-OptiPlex-3000:/home/exp5$ sudo gcc buffer.c -o buffer -pthread
siesgst@siesgst-OptiPlex-3000:/home/exp5$ sudo ./buffer
Producer produced item: 1
Producer inserted item 1 at index 0
Consumer removed item 1 from index 0
Consumer consumed item: 1
Producer produced item: 2
Producer inserted item 2 at index 1
Consumer removed item 2 from index 1
Consumer consumed item: 2
Producer produced item: 3
Producer inserted item 3 at index 2
Consumer removed item 3 from index 2
Consumer consumed item: 3
Producer produced item: 4
Producer inserted item 4 at index 3
Consumer removed item 4 from index 3
Consumer consumed item: 4
Program terminated after 4 iterations
siesgst@siesgst-OptiPlex-3000:/home/exp5$
```

Section B:

Part 1: Dry Run Table (BUFFER_SIZE = 3)
Assume:

- empty = 3, full = 0
- in = 0, out = 0
- Buffer shown as [_ _ _]

Step	Action	Buffer State	in	out	empty	full
0	Initial	[_ _ _]	0	0	3	0
1	Producer puts 1	[1 _ _]	1	0	2	1
2	Producer puts 2	[1 2 _]	2	0	1	2
3	Consumer takes	[_ 2 _]	2	1	2	1
4	Producer puts 3	[_ 2 3]	0	1	1	2
5	Consumer takes	[_ _ 3]	0	2	2	1

Part 2: Identify the Critical Section

Producer Critical Section

```
buffer[in] = item;
in = (in + 1) % BUFFER_SIZE;
```

Consumer Critical Section

```
item = buffer[out];
out = (out + 1) % BUFFER_SIZE;
```

Why protected?

- Prevents **race conditions** on buffer, in, and out
- Ensures **mutual exclusion**
- Avoids inconsistent reads/writes

Part 3: What-If Analysis

1. **If lock (mutex) is removed**
    - Race condition
    - Data corruption
    - Overwrite or wrong consumption
2. **If empty semaphore is not used**

- Producer may insert into a **full buffer**
- Buffer overflow

3. **If modulo operation is removed**
    - in / out exceed buffer size
    - Array index out of bounds

4. **If producer is faster than consumer**
    - Producer blocks when empty = 0
    - No overflow (correct behavior)

Part 4: Debug Checklist (Verified)

✔ Semaphore initialization correct
✔ No duplicate/missing wait() or signal()
✔ Critical section protected by mutex
✔ No buffer overflow / underflow
✔ No deadlock
✔ Correct order: wait → CS → signal

Part 5: Viva Questions:

1. **Why both empty and full semaphores?**
    - empty: prevents overflow
    - full: prevents underflow
2. **Counting vs Binary semaphore**
    - Binary: 0 or 1 (mutex)
    - Counting: range (buffer slots)
3. **Why mutex needed in addition to semaphores?**
    - Semaphores control quantity
    - Mutex protects shared data
4. **What is bounded buffer?**
    - Fixed-size buffer with limited capacity
5. **How does circular buffer help?**
    - Reuses space efficiently using modulo
6. **Does this solution prevent starvation?**
    - Yes, with fair semaphore scheduling

Section C:

In this variant, replace the mutex with a binary semaphore named 'lock'. Use counting semaphores 'empty' and 'full' for buffer capacity, and 'lock' for mutual exclusion. Students must implement the synchronization correctly using sem_wait/sem_post in the right order.

Conceptual Mapping:
• empty → counts empty slots in buffer • full → counts filled slots in buffer • lock → binary semaphore acting as a mutex (1 = free, 0 = locked)

Algorithm
(Semaphore-Only Variant)

Initialization:

1. Initialize empty = BUFFER_SIZE
2. Initialize full = 0
3. Initialize lock = 1 (binary semaphore)
4. Create producer and consumer threads

Producer (repeat):

1. Produce item
2. wait(empty)
3. wait(lock)
4. Insert item at buffer[in]
5. Update in circularly
6. signal(lock)
7. signal(full)

Consumer (repeat):

1. wait(full)
2. wait(lock)
3. Remove item from buffer[out]
4. Update out circularly
5. signal(lock)
6. signal(empty)

7. Consume item

Code:

```
  GNU nano 7.2                                                    buffer2.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 3
#define MAX_ITEMS 4

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
sem_t lock;

void* producer(void* arg) {
    int item = 1;

    for (int i = 0; i < MAX_ITEMS; i++) {
        printf("Producer produced item %d\n", item);

        sem_wait(&empty);
        sem_wait(&lock);

        buffer[in] = item;
        printf("Producer inserted %d at index %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&lock);
        sem_post(&full);

        item++;
        sleep(1);
    }

    pthread_exit(NULL);
}

void* consumer(void* arg) {
    int item;

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo      M-6
```

```
  GNU nano 7.2                                                    buffer2.c *

    for (int i = 0; i < MAX_ITEMS; i++) {
        sem_wait(&full);
        sem_wait(&lock);

        item = buffer[out];
        printf("Consumer removed %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&lock);
        sem_post(&empty);

        printf("Consumer consumed item %d\n", item);
        sleep(1);
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&lock, 0, 1);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&lock);

    printf("Program completed using semaphore-only synchronization\n");

    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo      M-6
```

```
siesgst@siesgst-OptiPlex-3000:/home/exp5$ sudo nano buffer2.c
siesgst@siesgst-OptiPlex-3000:/home/exp5$ sudo gcc buffer2.c -o buffer2 -pthread
siesgst@siesgst-OptiPlex-3000:/home/exp5$ sudo ./buffer2
Producer produced item 1
Producer inserted 1 at index 0
Consumer removed 1 from index 0
Consumer consumed item 1
Producer produced item 2
Producer inserted 2 at index 1
Consumer removed 2 from index 1
Consumer consumed item 2
Producer produced item 3
Producer inserted 3 at index 2
Consumer removed 3 from index 2
Consumer consumed item 3
Producer produced item 4
Producer inserted 4 at index 0
Consumer removed 4 from index 0
Consumer consumed item 4
Program completed using semaphore-only synchronization
siesgst@siesgst-OptiPlex-3000:/home/exp5$
```

Conclusion: This experiment provided a clear understanding of process synchronization using POSIX semaphores. By implementing the Producer–Consumer problem with a bounded buffer, the importance of semaphores in managing shared resources was demonstrated. The use of counting semaphores for buffer control and a binary semaphore for mutual exclusion effectively prevented race conditions. Overall, the experiment enhanced practical knowledge of multithreading, synchronization, and inter-thread communication in Linux system programming.